

Graduation Project Report

Smart Greenhouse Automation System

Project Title: Smart Greenhouse
Automation System

Department / Faculty:

.....

Supervised By:

.....

Team Members:

1.
2.
3.
4.

1. Abstract

This project presents a comprehensive Smart Greenhouse Automation System designed to monitor and control the internal environment of a greenhouse, ensuring optimal conditions for plant growth. The system architecture is divided into a hardware control layer and a mobile application interface layer. The hardware is powered by an ATmega32 microcontroller, which interfaces with various analog and digital sensors to acquire real-time environmental data (temperature, humidity, soil moisture, and light intensity). Based on state-machine logic, the microcontroller autonomously actuates relays controlling a water pump, a cooling fan, and LED lighting.

To provide real-time monitoring and manual override capabilities, a cross-platform mobile application was developed using .NET MAUI. The application communicates with the hardware via an HC-05 Bluetooth module using UART communication. The integration of continuous sensor polling without blocking delays (using hardware timers) ensures highly responsive remote control operations.

2. System Architecture & Block Diagram

The system consists of three main operational blocks: Sensory Input, Microcontroller Processing Unit, and Actuation/Communication Output.

- **Input Sensors:** DHT11 (Temperature & Humidity), Analog Soil Moisture Sensor, LDR (Light Dependent Resistor).
- **Control Unit:** ATmega32 operating at an 8MHz internal oscillator.
- **Output Actuators:** Active LOW Relays for Pump and Fan, and NPN Transistor-driven LEDs.
- **Communication:** HC-05 Bluetooth Module operating at 9600 baud rate.

3. Proteus Simulation & Hardware Wiring

The hardware implementation was verified using Proteus ISIS simulation prior to physical deployment. The wiring configuration is mapped as follows:

Component	ATmega32 Pin	Description & Specifications
Soil Moisture Sensor	PA0 (ADC0)	Analog input. Formula inverted in code: 0% represents dry, 100% represents submerged.
LDR Sensor	PA1	Configured with a 10k Ω pull-down resistor. HIGH indicates darkness.
DHT11 Sensor	PA2	Requires a 4.7k Ω pull-up resistor for stable One-Wire communication.
Water Pump Relay	PB0	Active LOW configuration. Driven by logic 0 to activate.
Cooling Fan Relay	PB1	Active LOW configuration. Activated at $\geq 30^{\circ}\text{C}$, deactivated at $\leq 27^{\circ}\text{C}$ (Hysteresis applied).
LED Lighting	PB2	Driven via a BC547 NPN Transistor with a 4.7k Ω base resistor to handle required current.
HC-05 Bluetooth (RXD)	PD1 (TXD)	UART Transmission line (Microcontroller to Bluetooth).
HC-05 Bluetooth (TXD)	PD0 (RXD)	UART Reception line (Bluetooth to Microcontroller).

4. Firmware Implementation (ATmega32 - Embedded C)

The following section details the complete firmware source code written in Embedded C for the ATmega32 microcontroller. The code is structured using a non-blocking state machine architecture.

4.1 main.c

```
/*=====
 * Smart Greenhouse – Main Entry Point
 * File: main.c
 *
 * ATmega32 @ 8MHz Internal RC Oscillator
 * Non-blocking super-loop with Timer0 for 1.5s scheduling
 *
 * Author: Mahmoud – Computer Engineering Dept.
 *=====*/
#include <avr/io.h>
#include <avr/interrupt.h>
#include "greenhouse_logic.h"
#include "hal_uart.h"

/*--- Timing ---*/
#define CYCLE_INTERVAL_MS 1500 /* Run state machine every 1.5 seconds */

volatile uint32_t g_millis = 0; /* Millisecond counter */

/*--- UART Command Buffer ---*/
#define CMD_BUF_SIZE 16
static char cmd_buf[CMD_BUF_SIZE];
static uint8_t cmd_idx = 0;

/*=====
 * Timer0 – Compare Match Interrupt (every 1ms)
 *
 * 8MHz / 64 prescaler = 125,000 ticks/sec
 * CTC mode: OCR0 = 124 → 125,000 / 125 = 1000 Hz = 1ms
 *=====*/
ISR(TIMER0_COMP_vect)
{
    g_millis++;
}

static void Timer0_Init(void)
{
    /* CTC mode (WGM01=1), prescaler = 64 (CS01+CS00=1) */
    TCCR0 = (1 << WGM01) | (1 << CS01) | (1 << CS00);
```

```

    /* Compare value for 1ms interrupt */
    OCR0 = 124;

    /* Enable Timer0 Compare Match interrupt */
    TIMSK |= (1 << OCIE0);
}

static uint32_t millis(void)
{
    uint32_t ms;
    cli();
    ms = g_millis;
    sei();
    return ms;
}

/*=====
 * UART Command Processing
 * Reads incoming bytes and buffers until '\n' is received
 *=====*/
static void process_uart_input(void)
{
    while (UART_Available()) {
        uint8_t c = UART_Receive();

        if (c == '\n' || c == '\r') {
            if (cmd_idx > 0) {
                cmd_buf[cmd_idx] = '\0';
                Greenhouse_ProcessCommand(cmd_buf);
                cmd_idx = 0;
            }
        } else {
            if (cmd_idx < CMD_BUF_SIZE - 1) {
                cmd_buf[cmd_idx++] = (char)c;
            }
        }
    }
}

/*=====
 * Main - Super Loop
 *=====*/
int main(void)
{
    /* Initialize all subsystems */
    Greenhouse_Init();
    Timer0_Init();

    /* Enable global interrupts */
    sei();

```

```

/* Send startup message */
UART_SendString("=== Smart Greenhouse v1.0 ===\r\n");

uint32_t last_cycle = 0;

/* Non-blocking main loop */
while (1) {
    /* Check for incoming commands from app */
    process_uart_input();

    /* Run state machine cycle every 1.5 seconds */
    uint32_t now = millis();
    if (now - last_cycle >= CYCLE_INTERVAL_MS) {
        last_cycle = now;

        /* Run all 5 states in sequence:
         * READ → EVALUATE → ACTUATE → TRANSMIT → IDLE */
        Greenhouse_Update(); /* IDLE → READ_SENSORS */
        Greenhouse_Update(); /* READ_SENSORS → EVALUATE */
        Greenhouse_Update(); /* EVALUATE → ACTUATE */
        Greenhouse_Update(); /* ACTUATE → TRANSMIT */
        Greenhouse_Update(); /* TRANSMIT → IDLE */
    }
}

return 0;
}

```

4.2 greenhouse_logic.c

```

/*=====
 * Smart Greenhouse – Automation Logic & State Machine
 * File: greenhouse_logic.c
 *
 * State Machine Flow:
 * IDLE → READ_SENSORS → EVALUATE → ACTUATE → TRANSMIT → IDLE
 *
 * Features:
 * - Hysteresis on thresholds to prevent relay chattering
 * - Manual override per actuator via UART commands including Light
 * - Non-blocking design (no delays in state machine)
 *=====*/
#include "greenhouse_logic.h"
#include "hal_adc.h"
#include "hal_uart.h"
#include "hal_dht11.h"
#include "hal_gpio.h"
#include <string.h>

/*--- Internal State ---*/
static SystemState    g_state = STATE_IDLE;

```

```

static GreenhouseData g_data = {0, 0, 0, 0, 0, 0, 0};
static OverrideMode g_pump_mode = MODE_AUTO;
static OverrideMode g_fan_mode = MODE_AUTO;
static OverrideMode g_led_mode = MODE_AUTO;

/* Last valid DHT11 readings (fallback if sensor fails) */
static uint8_t g_last_temp = 25;
static uint8_t g_last_hum = 50;

void Greenhouse_Init(void)
{
    ADC_Init();
    UART_Init();
    GPIO_Init();

    g_state = STATE_IDLE;
    g_pump_mode = MODE_AUTO;
    g_fan_mode = MODE_AUTO;
    g_led_mode = MODE_AUTO;
}

/*--- State: Read Sensors ---*/
static void state_read_sensors(void)
{
    /* Read soil moisture (ADC on PA0) */
    g_data.soil = ADC_ReadSoilPercent();

    /* Read DHT11 (PA2) - blocking ~25ms */
    DHT11_Data dht = DHT11_Read();
    if (dht.valid) {
        g_data.temperature = dht.temperature;
        g_data.humidity = dht.humidity;
        g_last_temp = dht.temperature;
        g_last_hum = dht.humidity;
    } else {
        /* Use last valid reading if sensor fails */
        g_data.temperature = g_last_temp;
        g_data.humidity = g_last_hum;
    }

    /* Read LDR (PA1) */
    g_data.light = GPIO_ReadLDR();
}

/*--- State: Evaluate & Decide ---*/
static void state_evaluate(void)
{
    /*
     * Pump Logic (with hysteresis):
     * - Turn ON when soil < 35%
     * - Turn OFF when soil > 50%
     * - Between 35-50%: maintain current state
    */

```

```

    */
    if (g_pump_mode == MODE_AUTO) {
        if (g_data.soil < SOIL_THRESHOLD_LOW) {
            g_data.pump_on = 1;
        } else if (g_data.soil > SOIL_THRESHOLD_HIGH) {
            g_data.pump_on = 0;
        }
        /* else: keep current state (hysteresis zone) */
    } else {
        g_data.pump_on = (g_pump_mode == MODE_MANUAL_ON) ? 1 : 0;
    }

    /*
    * Fan Logic (with hysteresis):
    * - Turn ON when temp > 45°C
    * - Turn OFF when temp < 40°C
    * - Between 40-45°C: maintain current state
    */
    if (g_fan_mode == MODE_AUTO) {
        if (g_data.temperature > TEMP_THRESHOLD_HIGH) {
            g_data.fan_on = 1;
        } else if (g_data.temperature < TEMP_THRESHOLD_LOW) {
            g_data.fan_on = 0;
        }
        /* else: keep current state (hysteresis zone) */
    } else {
        g_data.fan_on = (g_fan_mode == MODE_MANUAL_ON) ? 1 : 0;
    }

    /*
    * LED Logic: automatic by default (LDR based), overrideable via app
    * - Dark (LDR=1) → LED ON
    * - Light (LDR=0) → LED OFF
    */
    if (g_led_mode == MODE_AUTO) {
        g_data.led_on = g_data.light;
    } else {
        g_data.led_on = (g_led_mode == MODE_MANUAL_ON) ? 1 : 0;
    }
}

/*--- State: Actuate Outputs ---*/
static void state_actuate(void)
{
    /* Respect configured relay polarity */
    #if RELAY_ACTIVE_LOW
        /* Active LOW: LOW turns relay ON, HIGH turns OFF */
        if (g_data.pump_on) GPIO_SetLow(PUMP_PIN);
        else GPIO_SetHigh(PUMP_PIN);

        if (g_data.fan_on) GPIO_SetLow(FAN_PIN);
        else GPIO_SetHigh(FAN_PIN);
    #endif
}

```



```

#else
    /* Active HIGH: HIGH turns relay ON, LOW turns OFF */
    if (g_data.pump_on) GPIO_SetHigh(PUMP_PIN);
    else GPIO_SetLow(PUMP_PIN);

    if (g_data.fan_on) GPIO_SetHigh(FAN_PIN);
    else GPIO_SetLow(FAN_PIN);
#endif

    /* LED (direct drive, active HIGH) */
    if (g_data.led_on)
        GPIO_SetHigh(LED_PIN);
    else
        GPIO_SetLow(LED_PIN);
}

/*--- State: Transmit Data via UART ---*/
static void state_transmit(void)
{
    /* Format: S:{soil}%,T:{temp}C,H:{hum}%,L:{light},P:{pump},F:{fan}\r\n */

    UART_SendString("S:");
    UART_SendInt(g_data.soil);
    UART_SendString("%,T:");
    UART_SendInt(g_data.temperature);
    UART_SendString("C,H:");
    UART_SendInt(g_data.humidity);
    UART_SendString("%,L:");
    UART_SendString(g_data.led_on ? "ON" : "OFF");
    UART_SendString(",P:");
    UART_SendString(g_data.pump_on ? "ON" : "OFF");
    UART_SendString(",F:");
    UART_SendString(g_data.fan_on ? "ON" : "OFF");
    UART_SendString("\r\n");
}

/*--- Main State Machine Update ---*/
void Greenhouse_Update(void)
{
    switch (g_state) {
        case STATE_IDLE:
            g_state = STATE_READ_SENSORS;
            break;

        case STATE_READ_SENSORS:
            state_read_sensors();
            g_state = STATE_EVALUATE;
            break;

        case STATE_EVALUATE:
            state_evaluate();
            g_state = STATE_ACTUATE;

```

```

        break;

    case STATE_ACTUATE:
        state_actuate();
        g_state = STATE_TRANSMIT;
        break;

    case STATE_TRANSMIT:
        state_transmit();
        g_state = STATE_IDLE;
        break;

    default:
        g_state = STATE_IDLE;
        break;
}
}

GreenhouseData Greenhouse_GetData(void)
{
    return g_data;
}

/*--- Process Override Commands from App ---*/
void Greenhouse_ProcessCommand(const char *cmd)
{
    /* Commands: "P:ON", "P:OFF", "F:ON", "F:OFF", "L:ON", "L:OFF", "A:AUTO" */

    if (cmd[0] == 'P' && cmd[1] == ':') {
        if (cmd[2] == 'O' && cmd[3] == 'N') {
            g_pump_mode = MODE_MANUAL_ON;
        } else if (cmd[2] == 'O' && cmd[3] == 'F') {
            g_pump_mode = MODE_MANUAL_OFF;
        }
    }
    else if (cmd[0] == 'F' && cmd[1] == ':') {
        if (cmd[2] == 'O' && cmd[3] == 'N') {
            g_fan_mode = MODE_MANUAL_ON;
        } else if (cmd[2] == 'O' && cmd[3] == 'F') {
            g_fan_mode = MODE_MANUAL_OFF;
        }
    }
    else if (cmd[0] == 'L' && cmd[1] == ':') {
        if (cmd[2] == 'O' && cmd[3] == 'N') {
            g_led_mode = MODE_MANUAL_ON;
        } else if (cmd[2] == 'O' && cmd[3] == 'F') {
            g_led_mode = MODE_MANUAL_OFF;
        }
    }
    else if (cmd[0] == 'A' && cmd[1] == ':') {
        /* Return all to automatic */
        g_pump_mode = MODE_AUTO;
    }
}

```

```

        g_fan_mode  = MODE_AUTO;
        g_led_mode  = MODE_AUTO;
    }
}

```

4.3 greenhouse_logic.h

```

/*=====
 * Smart Greenhouse – Automation Logic & State Machine
 * File: greenhouse_logic.h
 *=====*/
#ifndef GREENHOUSE_LOGIC_H
#define GREENHOUSE_LOGIC_H

#include <stdint.h>

/*--- Automation Thresholds ---*/
#define SOIL_THRESHOLD_LOW    35    /* Below 35% → Pump ON (soil is dry) */
#define SOIL_THRESHOLD_HIGH  50    /* Above 50% → Pump OFF (soil is wet) */
#define TEMP_THRESHOLD_HIGH  30    /* Above 30°C → Fan ON */
#define TEMP_THRESHOLD_LOW   27    /* Below 27°C → Fan OFF (hysteresis) */

/*--- Hardware Config ---*/
#define RELAY_ACTIVE_LOW     1     /* Set to 0 for Active HIGH relays, 1 for Active LOW */

/*--- Override Modes ---*/
typedef enum {
    MODE_AUTO,          /* Automatic control (default) */
    MODE_MANUAL_ON,     /* Manually forced ON */
    MODE_MANUAL_OFF     /* Manually forced OFF */
} OverrideMode;

/*--- State Machine States ---*/
typedef enum {
    STATE_IDLE,
    STATE_READ_SENSORS,
    STATE_EVALUATE,
    STATE_ACTUATE,
    STATE_TRANSMIT
} SystemState;

/*--- Live Sensor Data ---*/
typedef struct {
    uint8_t soil;        /* Soil moisture (0-100%) */
    uint8_t temperature; /* Temperature (°C) */
    uint8_t humidity;    /* Humidity (%) */
    uint8_t light;       /* LDR: 1=dark, 0=light */
    uint8_t pump_on;     /* Pump status: 1=ON, 0=OFF */
    uint8_t fan_on;      /* Fan status: 1=ON, 0=OFF */
    uint8_t led_on;      /* LED status: 1=ON, 0=OFF */
} GreenhouseData;

```

```
/*--- Public API ---*/

/* Initialize greenhouse logic */
void Greenhouse_Init(void);

/* Run one cycle of the state machine (call from main loop) */
void Greenhouse_Update(void);

/* Get current sensor data (for display/debug) */
GreenhouseData Greenhouse_GetData(void);

/* Process incoming UART command string (e.g., "P:ON", "F:OFF", "A:AUTO") */
void Greenhouse_ProcessCommand(const char *cmd);

#endif /* GREENHOUSE_LOGIC_H */
```

5. Mobile Application Implementation (.NET MAUI)

The mobile application was built using the MVVM (Model-View-ViewModel) architectural pattern to ensure clean separation between UI components and backend logic.

5.1 DashboardViewModel.cs

```
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows.Input;

namespace SmartGreenhouseApp.ViewModels;

public class DashboardViewModel : INotifyPropertyChanged
{
    private readonly Services.BluetoothService _bluetooth;
    public event PropertyChangedEventHandler? PropertyChanged;

    // --- Sensor Properties ---
    private int _soilMoisture;
    public int SoilMoisture
    {
        get => _soilMoisture;
        set { _soilMoisture = value; OnPropertyChanged();
OnPropertyChanged(nameof(SoilColor)); }
    }

    private int _temperature;
    public int Temperature
    {
        get => _temperature;
        set { _temperature = value; OnPropertyChanged();
OnPropertyChanged(nameof(TempColor)); }
    }

    private int _humidity;
    public int Humidity
    {
        get => _humidity;
        set { _humidity = value; OnPropertyChanged();
OnPropertyChanged(nameof(HumidityProgress)); }
    }

    private bool _isLightOn;
    public bool IsLightOn
    {
        get => _isLightOn;
        set { _isLightOn = value; OnPropertyChanged();
```

```

OnPropertyChanged(nameof(LightStatusText)); OnPropertyChanged(nameof(LightStatusColor)); }
    }

    private bool _isPumpOn;
    public bool IsPumpOn
    {
        get => _isPumpOn;
        set { _isPumpOn = value; OnPropertyChanged();
OnPropertyChanged(nameof(PumpStatusText)); OnPropertyChanged(nameof(PumpStatusColor));
OnPropertyChanged(nameof(PumpButtonText)); }
    }

    private bool _isFanOn;
    public bool IsFanOn
    {
        get => _isFanOn;
        set { _isFanOn = value; OnPropertyChanged();
OnPropertyChanged(nameof(FanStatusText)); OnPropertyChanged(nameof(FanStatusColor));
OnPropertyChanged(nameof(FanButtonText)); }
    }

    // --- Connection Properties ---
    private bool _isConnected;
    public bool IsConnected
    {
        get => _isConnected;
        set { _isConnected = value; OnPropertyChanged();
OnPropertyChanged(nameof(ConnectionColor)); OnPropertyChanged(nameof(ConnectButtonText)); }
    }

    private string _statusMessage = "🖱 Enter COM port (e.g. COM3) and press Connect";
    public string StatusMessage
    {
        get => _statusMessage;
        set { _statusMessage = value; OnPropertyChanged(); }
    }

    private string _portEntry = "";
    public string PortEntry
    {
        get => _portEntry;
        set { _portEntry = value; OnPropertyChanged(); }
    }

    private List<string> _availablePorts = new();
    public List<string> AvailablePorts
    {
        get => _availablePorts;
        set { _availablePorts = value; OnPropertyChanged(); }
    }

    private string? _selectedPort;

```

```

public string? SelectedPort
{
    get => _selectedPort;
    set { _selectedPort = value; OnPropertyChanged(); }
}

// --- Computed Properties ---
public string PumpStatusText => IsPumpOn ? "ON" : "OFF";
public string FanStatusText => IsFanOn ? "ON" : "OFF";
public string LightStatusText => IsLightOn ? "ON" : "OFF";
public double HumidityProgress => Humidity / 100.0;
public Color PumpStatusColor => IsPumpOn ? Color.FromArgb("#2ECC71") :
Color.FromArgb("#E74C3C");
    public Color FanStatusColor => IsFanOn ? Color.FromArgb("#2ECC71") :
Color.FromArgb("#E74C3C");
    public Color LightStatusColor => IsLightOn ? Color.FromArgb("#F39C12") :
Color.FromArgb("#636E72");
    public Color ConnectionColor => IsConnected ? Color.FromArgb("#2ECC71") :
Color.FromArgb("#E74C3C");
    public string ConnectButtonText => IsConnected ? "Disconnect" : "Connect";
    public string PumpButtonText => IsPumpOn ? "■ Pump OFF" : "► Pump ON";
    public string FanButtonText => IsFanOn ? "■ Fan OFF" : "► Fan ON";
    public string LightButtonText => IsLightOn ? "■ Light OFF" : "► Light ON";

public Color SoilColor
{
    get
    {
        if (SoilMoisture < 35) return Color.FromArgb("#E74C3C");
        if (SoilMoisture < 60) return Color.FromArgb("#F39C12");
        return Color.FromArgb("#2ECC71");
    }
}

public Color TempColor
{
    get
    {
        if (Temperature >= 45) return Color.FromArgb("#E74C3C");
        if (Temperature >= 35) return Color.FromArgb("#F39C12");
        return Color.FromArgb("#3498DB");
    }
}

// --- Commands ---
public ICommand ConnectCommand { get; }
public ICommand TogglePumpCommand { get; }
public ICommand ToggleFanCommand { get; }
public ICommand ToggleLightCommand { get; }
public ICommand ResetAutoCommand { get; }
public ICommand RefreshPortsCommand { get; }

```

```

public DashboardViewModel(Services.BluetoothService bluetooth)
{
    _bluetooth = bluetooth;

    _bluetooth.OnDataReceived += data =>
    {
        MainThread.BeginInvokeOnMainThread(() =>
        {
            SoilMoisture = data.SoilMoisture;
            Temperature = data.Temperature;
            Humidity = data.Humidity;
            IsLightOn = data.IsLightOn;
            IsPumpOn = data.IsPumpOn;
            IsFanOn = data.IsFanOn;
            StatusMessage = $"🌱 Live - Soil:{data.SoilMoisture}% Temp:
{data.Temperature}°C - {data.Timestamp:HH:mm:ss}";
        });
    };

    _bluetooth.OnStatusChanged += msg =>
        MainThread.BeginInvokeOnMainThread(() => StatusMessage = msg);

    _bluetooth.OnError += msg =>
        MainThread.BeginInvokeOnMainThread(() => StatusMessage = $"⚠️ {msg}");

    ConnectCommand = new Command(async () => await ToggleConnectionAsync());
    TogglePumpCommand = new Command(() => TogglePump());
    ToggleFanCommand = new Command(() => ToggleFan());
    ToggleLightCommand = new Command(() => ToggleLight());
    ResetAutoCommand = new Command(() => ResetAuto());
    RefreshPortsCommand = new Command(async () => await RefreshPortsAsync());

    // Perform initial check gracefully without locking UI
    _ = RefreshPortsAsync();
}

private async Task RefreshPortsAsync()
{
    #if ANDROID
    try
    {
        var permissionStatus = await Permissions.CheckStatusAsync<Permissions.Bluetooth>
();
        if (permissionStatus != PermissionStatus.Granted)
        {
            MainThread.BeginInvokeOnMainThread(() => StatusMessage = "🔒 Requesting
Bluetooth permissions...");
            permissionStatus = await Permissions.RequestAsync<Permissions.Bluetooth>();
            if (permissionStatus != PermissionStatus.Granted)
            {
                MainThread.BeginInvokeOnMainThread(() => StatusMessage = "⚠️ Bluetooth
permission denied. Cannot discover devices.");
            }
        }
    }
    #endif
}

```



```

        return;
    }
}
}
catch (Exception ex)
{
    MainThread.BeginInvokeOnMainThread(() => StatusMessage = $"⚠ Permission check
failed: {ex.Message}");
}
#endif

var ports = _bluetooth.GetAvailablePorts();
MainThread.BeginInvokeOnMainThread(() =>
{
    AvailablePorts = ports;
    if (AvailablePorts.Count > 0)
    {
        SelectedPort = AvailablePorts[0];
        StatusMessage = $"🔍 Found: {string.Join(", ", AvailablePorts)} – select and
connect";
    }
    else
    {
        StatusMessage = $"📌 Type COM port below (e.g. COM3) then press Connect";
    }
});
}

private async Task ToggleConnectionAsync()
{
    if (IsConnected)
    {
        _bluetooth.Disconnect();
        IsConnected = false;
        return;
    }

#if ANDROID
    try
    {
        var permissionStatus = await Permissions.CheckStatusAsync<Permissions.Bluetooth>
();

        if (permissionStatus != PermissionStatus.Granted)
        {
            StatusMessage = $"🔒 Granting Bluetooth permissions required to connect...";
            permissionStatus = await Permissions.RequestAsync<Permissions.Bluetooth>();
            if (permissionStatus != PermissionStatus.Granted)
            {
                StatusMessage = $"⚠ Permission denied. Connection cancelled.";
                return;
            }
        }
    }
}

```

```

    }
    catch (Exception ex)
    {
        StatusMessage = $"⚠ Setup error: {ex.Message}";
        return;
    }
#endif

    // Use typed port or selected port
    string? port = !string.IsNullOrEmpty(PortEntry) ? PortEntry.Trim().ToUpper() :
SelectedPort;

    if (string.IsNullOrEmpty(port))
    {
        StatusMessage = "⚠ Select a device or type a port name first.";
        return;
    }

    // Auto-add COM prefix if user just typed a number on Windows targets
    if (int.TryParse(port, out _))
        port = "COM" + port;

    StatusMessage = $"⌚ Connecting to {port}...";
    bool success = await _bluetooth.ConnectAsync(port);
    IsConnected = success;
    if (!success)
        StatusMessage = $"❌ Failed – check connection to {port}";
}

private void TogglePump()
{
    string cmd = IsPumpOn ? "P:OFF" : "P:ON";
    _bluetooth.SendCommand(cmd);
}

private void ToggleFan()
{
    string cmd = IsFanOn ? "F:OFF" : "F:ON";
    _bluetooth.SendCommand(cmd);
}

private void ToggleLight()
{
    string cmd = IsLightOn ? "L:OFF" : "L:ON";
    _bluetooth.SendCommand(cmd);
}

private void ResetAuto()
{
    _bluetooth.SendCommand("A:AUTO");
    StatusMessage = $"🔄 Switched to AUTO mode";
}

```

```

        protected void OnPropertyChanged([CallerMemberName] string? name = null)
            => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }

```

5.2 DataParser.cs

```

using System.Text;

namespace SmartGreenhouseApp.Services;

/// <summary>
/// Stateful buffer-based parser for Bluetooth serial stream.
/// Handles packet fragmentation – accumulates bytes until \r\n delimiter.
/// </summary>
public class DataParser
{
    private readonly StringBuilder _buffer = new();
    private const int MaxBufferSize = 512;

    /// <summary>
    /// Fired when a complete, valid data line is parsed.
    /// </summary>
    public event Action<Models.SensorData>? OnDataParsed;

    /// <summary>
    /// Feed incoming raw bytes from Bluetooth into the parser.
    /// May fire OnDataParsed zero or more times.
    /// </summary>
    public void Feed(string chunk)
    {
        _buffer.Append(chunk);

        // Prevent memory leak from garbage data
        if (_buffer.Length > MaxBufferSize)
        {
            _buffer.Clear();
            return;
        }

        // Process all complete lines
        while (true)
        {
            string current = _buffer.ToString();
            int delimIndex = current.IndexOf('\n');

            if (delimIndex < 0)
                break;

            // Extract the line (strip \r\n)
            string line = current.Substring(0, delimIndex).TrimEnd('\r');

```

```

        _buffer.Remove(0, delimIndex + 1);

        // Validate: must contain at least "S:" and "T:"
        if (line.Contains("S:") && line.Contains("T:"))
        {
            var data = Models.SensorData.Parse(line);
            OnDataParsed?.Invoke(data);
        }
    }
}

/// <summary>
/// Reset the internal buffer.
/// </summary>
public void Reset()
{
    _buffer.Clear();
}
}

```

5.3 DashboardPage.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:vm="clr-namespace:SmartGreenhouseApp.ViewModels"
    x:Class="SmartGreenhouseApp.Views.DashboardPage"
    x:DataType="vm:DashboardViewModel"
    Title="Smart Greenhouse"
    BackgroundColor="#0D1117">

    <ScrollView>
        <VerticalStackLayout Padding="20" Spacing="14">

            <!-- HEADER -->
            <Grid ColumnDefinitions="Auto,*,Auto" Padding="0,0,0,4">
                <Label Text="🌱" FontSize="28" VerticalOptions="Center"/>
                <VerticalStackLayout Grid.Column="1" Padding="12,0">
                    <Label Text="Smart Greenhouse" FontSize="22" FontAttributes="Bold"
                        TextColor="White"/>
                    <Label Text="{Binding StatusMessage}" FontSize="11" TextColor="#8B95A5"
                        LineBreakMode="TailTruncation"/>
                </VerticalStackLayout>
                <Ellipse Grid.Column="2" WidthRequest="14" HeightRequest="14"
                    Fill="{Binding ConnectionColor}" VerticalOptions="Center"/>
            </Grid>

            <!-- CONNECTION -->
            <Frame BackgroundColor="#161B22" BorderColor="#30363D" CornerRadius="12"
                Padding="14" HasShadow="False">
                <VerticalStackLayout Spacing="10">

```

```

<!-- Dropdown for detected ports -->
<Grid ColumnDefinitions="*,Auto" ColumnSpacing="8">
    <Picker Title="Detected Ports"
        ItemsSource="{Binding AvailablePorts}"
        SelectedItem="{Binding SelectedPort}"
        TextColor="White" TitleColor="#636E72"
        BackgroundColor="#0D1117"/>
    <Button Grid.Column="1" Text="↻"
        Command="{Binding RefreshPortsCommand}"
        BackgroundColor="#30363D" TextColor="White"
        CornerRadius="8" WidthRequest="44" HeightRequest="44"
FontSize="18"/>
</Grid>
<!-- Manual entry + Connect -->
<Grid ColumnDefinitions="*,Auto" ColumnSpacing="8">
    <Entry Placeholder="Or type: COM3, COM5..."
        Text="{Binding PortEntry}"
        TextColor="White" PlaceholderColor="#636E72"
        BackgroundColor="#0D1117"
        FontSize="14"/>
    <Button Grid.Column="1"
        Text="{Binding ConnectButtonText}"
        Command="{Binding ConnectCommand}"
        BackgroundColor="{Binding ConnectionColor}"
        TextColor="White" CornerRadius="8"
        FontAttributes="Bold" HeightRequest="44"
        MinimumWidthRequest="110"/>
</Grid>
</VerticalStackLayout>
</Frame>

<!-- GAUGES -->
<Grid ColumnDefinitions="*,*" ColumnSpacing="12">
    <Frame BackgroundColor="#161B22" BorderColor="#30363D" CornerRadius="16"
Padding="10" HasShadow="False">
        <VerticalStackLayout Spacing="4">
            <Label Text="🌡 Temperature" FontSize="13" TextColor="#8B95A5"
HorizontalOptions="Center"/>
            <GraphicsView x:Name="TempGauge" WidthRequest="140"
HeightRequest="140" HorizontalOptions="Center"/>
            <Label Text="{Binding Temperature, StringFormat='{0}°C'}"
FontSize="16" TextColor="{Binding TempColor}"
HorizontalOptions="Center" FontAttributes="Bold"/>
        </VerticalStackLayout>
    </Frame>
    <Frame Grid.Column="1" BackgroundColor="#161B22" BorderColor="#30363D"
CornerRadius="16" Padding="10" HasShadow="False">
        <VerticalStackLayout Spacing="4">
            <Label Text="💧 Soil Moisture" FontSize="13" TextColor="#8B95A5"
HorizontalOptions="Center"/>
            <GraphicsView x:Name="SoilGauge" WidthRequest="140"
HeightRequest="140" HorizontalOptions="Center"/>

```

```

        <Label Text="{Binding SoilMoisture, StringFormat='{0}%'}"
            FontSize="16" TextColor="{Binding SoilColor}"
            HorizontalOptions="Center" FontAttributes="Bold"/>
    </VerticalStackLayout>
</Frame>
</Grid>

<!-- HUMIDITY -->
<Frame BackgroundColor="#161B22" BorderColor="#30363D" CornerRadius="12"
Padding="14" HasShadow="False">
    <Grid ColumnDefinitions="Auto,*,Auto">
        <Label Text="☹️" FontSize="24" VerticalOptions="Center"/>
        <VerticalStackLayout Grid.Column="1" Padding="12,0">
            <Label Text="Humidity" FontSize="14" TextColor="#8B95A5"/>
            <Label Text="{Binding Humidity, StringFormat='{0}%'}"
                FontSize="22" TextColor="White" FontAttributes="Bold"/>
        </VerticalStackLayout>
        <ProgressBar Grid.Column="2" Progress="{Binding HumidityProgress}"
            ProgressColor="#3498DB" WidthRequest="80"
VerticalOptions="Center"/>
    </Grid>
</Frame>

<!-- STATUS -->
<Label Text="System Status" FontSize="16" TextColor="#8B95A5"
FontAttributes="Bold"/>
    <Grid ColumnDefinitions="*,*,*" ColumnSpacing="8">
        <Frame BackgroundColor="#161B22" BorderColor="#30363D" CornerRadius="12"
Padding="10" HasShadow="False">
            <VerticalStackLayout Spacing="4" HorizontalOptions="Center">
                <Label Text="🔌" FontSize="24" HorizontalOptions="Center"/>
                <Label Text="Pump" FontSize="12" TextColor="#8B95A5"
HorizontalOptions="Center"/>
            </VerticalStackLayout>
            <Frame BackgroundColor="{Binding PumpStatusColor}" CornerRadius="10"
Padding="8,3" HasShadow="False" HorizontalOptions="Center">
                <Label Text="{Binding PumpStatusText}" FontSize="12"
TextColor="White" FontAttributes="Bold"/>
            </Frame>
        </Grid>
        <Frame Grid.Column="1" BackgroundColor="#161B22" BorderColor="#30363D"
CornerRadius="12" Padding="10" HasShadow="False">
            <VerticalStackLayout Spacing="4" HorizontalOptions="Center">
                <Label Text="🌀" FontSize="24" HorizontalOptions="Center"/>
                <Label Text="Fan" FontSize="12" TextColor="#8B95A5"
HorizontalOptions="Center"/>
            </VerticalStackLayout>
            <Frame BackgroundColor="{Binding FanStatusColor}" CornerRadius="10"
Padding="8,3" HasShadow="False" HorizontalOptions="Center">
                <Label Text="{Binding FanStatusText}" FontSize="12"
TextColor="White" FontAttributes="Bold"/>
            </Frame>
        </Grid>
    </Grid>

```

```

        </Frame>
        <Frame Grid.Column="2" BackgroundColor="#161B22" BorderColor="#30363D"
        CornerRadius="12" Padding="10" HasShadow="False">
            <VerticalStackLayout Spacing="4" HorizontalOptions="Center">
                <Label Text="💡" FontSize="24" HorizontalOptions="Center"/>
                <Label Text="Light" FontSize="12" TextColor="#8B95A5"
                HorizontalOptions="Center"/>
                <Frame BackgroundColor="{Binding LightStatusColor}"
                CornerRadius="10" Padding="8,3" HasShadow="False" HorizontalOptions="Center">
                    <Label Text="{Binding LightStatusText}" FontSize="12"
                    TextColor="White" FontAttributes="Bold"/>
                </Frame>
            </VerticalStackLayout>
        </Frame>
    </Grid>

    <!-- CONTROLS -->
    <Label Text="Manual Override" FontSize="16" TextColor="#8B95A5"
    FontAttributes="Bold"/>
    <Grid ColumnDefinitions="*,*,*" ColumnSpacing="10">
        <Button Text="{Binding PumpButtonText}" Command="{Binding
        TogglePumpCommand}"
            BackgroundColor="#1A5276" TextColor="White" CornerRadius="12"
            FontAttributes="Bold" HeightRequest="48" FontSize="12"/>
        <Button Grid.Column="1" Text="{Binding FanButtonText}" Command="{Binding
        ToggleFanCommand}"
            BackgroundColor="#1A5276" TextColor="White" CornerRadius="12"
            FontAttributes="Bold" HeightRequest="48" FontSize="12"/>
        <Button Grid.Column="2" Text="{Binding LightButtonText}" Command="{Binding
        ToggleLightCommand}"
            BackgroundColor="#7D6608" TextColor="White" CornerRadius="12"
            FontAttributes="Bold" HeightRequest="48" FontSize="12"/>
    </Grid>
    <Button Text="🔄 Reset to AUTO Mode" Command="{Binding ResetAutoCommand}"
        BackgroundColor="#2ECC71" TextColor="White" CornerRadius="12"
        FontAttributes="Bold" HeightRequest="46" FontSize="14"/>

    </VerticalStackLayout>
</ScrollView>
</ContentPage>

```